```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
%matplotlib inline
plt.rcParams['figure.figsize'] = (15, 5)
plt.rcParams['axes.grid'] = True

# Plotly imports
import plotly.offline as iplot
import plotly.graph_objs as go
import plotly.offline import init_notebook_mode, iplot, plot
init_notebook_mode(connected=True)

# Cufflinks imports
import cufflinks as cf
cf.go_offline()

# Set warnings to ignore
warnings.filterwarnings('ignore')

# Displaying all columns
pd.set_option('display.max_columns', None)
```

```
# Data loading
df = pd.read_csv('/kaggle/input/portfolio-analysis-yahoo-finance/portfolio.csv')
df['Date'] = pd.to_datetime(df['Date'], utc=True)
df.shape

# Checking column dtypes and columns
df.columns, df.dtypes

# Checking if duplicate exists(False means good)
df.duplicated().sum()

# Checking missing values
df.isnull().sum()

# Checking unique values
df.nunique()

# Checking unique values
df['Ticker'].unique()

# Checking value counts
df['Ticker'].value_counts()

# Checking data info
df.info()

# Checking data description
df.describe()

# Checking data description for categorical columns
df.describe(include=['object'])

# Checking data description for all columns
df.describe(include='all')

# Checking data sample
df.sample(5)

# Checking data head
df.head()

# Checking data tail
df.tail()
```

```
# Filtering data for specific tickers
tickers = ['AAPL', 'AMZN', 'TSLA', 'JPM', 'PG']
df = df[df['Ticker'].isin(tickers)]

# Pivot table to reshape data for analysis
pivot_df = df.pivot_table(index='Date', columns='Ticker', values=['Close', 'Volume'])

# Calculate daily returns
daily_returns = pivot_df['Close'].pct_change()

# Calculate cumulative returns
cumulative_returns = (1 + daily_returns).cumprod()

# Calculate rolling 30-day volatility
rolling_volatility = daily_returns.rolling(window=30).std()

# Calculate correlation between stocks
correlation_matrix = daily_returns.corr()

# Calculate portfolio weights (equal weight for simplicity)
weights = pd.Series(1/len(tickers), index=tickers)

# Calculate portfolio returns
portfolio_returns = (daily_returns * weights).sum(axis=1)

# Calculate portfolio cumulative returns
portfolio_cumulative = (1 + portfolio_returns).cumprod()

# Calculate portfolio volatility
portfolio_volatility = portfolio_returns.rolling(window=30).std()

# Calculate Sharpe ratio (assuming risk-free rate of 0.02)
risk_free_rate = 0.02
sharpe_ratio = (portfolio_returns.mean() * 252 - risk_free_rate) / (portfolio_returns.std() * np.sqrt(252))

# Calculate maximum drawdown
def calculate_max_drawdown(returns):
    cumulative = (1 + returns).cumprod()
    peak = cumulative.expanding(min_periods=1).max()
    drawdown = (cumulative / peak) - 1
    return drawdown.min()

max_drawdown = calculate_max_drawdown(portfolio_returns)

# Calculate Value at Risk (VaR) at 95% confidence level
var_95 = np.percentile(portfolio_returns, 5)

# Calculate Conditional VaR (CVaR)
cvar_95 = portfolio_returns[portfolio_returns <= var_95].mean()

# Calculate beta (assuming market return is SPY)
market_returns = daily_returns['SPY']
beta = portfolio_returns.cov(market_returns) / market_returns.var()

# Calculate alpha
alpha = portfolio_returns.mean() * 252 - risk_free_rate - beta * (market_returns.mean() * 252 - risk_free_rate)

# Calculate information ratio
tracking_error = (portfolio_returns - market_returns).std() * np.sqrt(252)
information_ratio = (portfolio_returns.mean() - market_returns.mean()) * 252 / tracking_error

# Calculate Sortino ratio
downside_returns = portfolio_returns[portfolio_returns < 0]
sortino_ratio = (portfolio_returns.mean() * 252 - risk_free_rate) / (downside_returns.std() * np.sqrt(252))

# Calculate Calmar ratio
calmar_ratio = portfolio_returns.mean() * 252 / abs(max_drawdown)

# Calculate monthly returns
monthly_returns = portfolio_returns.resample('M').apply(lambda x: (1 + x).prod() - 1)

# Calculate quarterly returns
quarterly_returns = portfolio_returns.resample('Q').apply(lambda x: (1 + x).prod() - 1)

# Calculate annual returns
annual_returns = portfolio_returns.resample('Y').apply(lambda x: (1 + x).prod() - 1)

# Calculate rolling 252-day (1 year) Sharpe ratio
rolling_sharpe = (portfolio_returns.rolling(window=252).mean() * 252 - risk_free_rate) / (portfolio_returns.rolling(window=252).std() * np.sqrt(252))

# Calculate rolling 252-day (1 year) maximum drawdown
def rolling_max_drawdown(returns, window):
    return returns.rolling(window=window).apply(lambda x: calculate_max_drawdown(x))

rolling_max_dd = rolling_max_drawdown(portfolio_returns, 252)

# Calculate rolling 252-day (1 year) VaR
rolling_var = portfolio_returns.rolling(window=252).apply(lambda x: np.percentile(x, 5))

# Calculate rolling 252-day (1 year) CVaR
def rolling_cvar(returns, window):
    return returns.rolling(window=window).apply(lambda x: x[x <= np.percentile(x, 5)].mean())

rolling_cvar = rolling_cvar(portfolio_returns, 252)

# Calculate rolling 252-day (1 year) beta
rolling_beta = portfolio_returns.rolling(window=252).cov(market_returns) / market_returns.rolling(window=252).var()

# Calculate rolling 252-day (1 year) alpha
rolling_alpha = portfolio_returns.rolling(window=252).mean() * 252 - risk_free_rate - rolling_beta * (market_returns.rolling(window=252).mean() * 252 - risk_free_rate)

# Calculate rolling 252-day (1 year) information ratio
rolling_information = (portfolio_returns.rolling(window=252).mean() - market_returns.rolling(window=252).mean()) * 252 / (portfolio_returns - market_returns).rolling(window=252).std() * np.sqrt(252)

# Calculate rolling 252-day (1 year) Sortino ratio
def rolling_sortino(returns, window):
    downside = returns[returns < 0]
    return (returns.rolling(window=window).mean() * 252 - risk_free_rate) / (downside.rolling(window=window).std() * np.sqrt(252))

rolling_sortino = rolling_sortino(portfolio_returns, 252)

# Calculate rolling 252-day (1 year) Calmar ratio
rolling_calmar = portfolio_returns.rolling(window=252).mean() * 252 / abs(rolling_max_dd)

# Calculate rolling 252-day (1 year) correlation matrix
def rolling_correlation(df, window):
    return df.rolling(window=window).corr()

rolling_correlation = rolling_correlation(daily_returns, 252)

# Calculate rolling 252-day (1 year) volatility
rolling_volatility = daily_returns.rolling(window=252).std() * np.sqrt(252)

# Calculate rolling 252-day (1 year) Sharpe ratio
rolling_sharpe = (daily_returns.rolling(window=252).mean() * 252 - risk_free_rate) / (daily_returns.rolling(window=252).std() * np.sqrt(252))

# Calculate rolling 252-day (1 year) maximum drawdown
def rolling_max_drawdown(returns, window):
    return returns.rolling(window=window).apply(lambda x: calculate_max_drawdown(x))

rolling_max_dd = rolling_max_drawdown(daily_returns, 252)

# Calculate rolling 252-day (1 year) VaR
rolling_var = daily_returns.rolling(window=252).apply(lambda x: np.percentile(x, 5))

# Calculate rolling 252-day (1 year) CVaR
def rolling_cvar(returns, window):
    return returns.rolling(window=window).apply(lambda x: x[x <= np.percentile(x, 5)].mean())

rolling_cvar = rolling_cvar(daily_returns, 252)

# Calculate rolling 252-day (1 year) beta
rolling_beta = daily_returns.rolling(window=252).cov(market_returns) / market_returns.rolling(window=252).var()

# Calculate rolling 252-day (1 year) alpha
rolling_alpha = daily_returns.rolling(window=252).mean() * 252 - risk_free_rate - rolling_beta * (market_returns.rolling(window=252).mean() * 252 - risk_free_rate)

# Calculate rolling 252-day (1 year) information ratio
rolling_information = (daily_returns.rolling(window=252).mean() - market_returns.rolling(window=252).mean()) * 252 / (daily_returns - market_returns).rolling(window=252).std() * np.sqrt(252)

# Calculate rolling 252-day (1 year) Sortino ratio
def rolling_sortino(returns, window):
    downside = returns[returns < 0]
    return (returns.rolling(window=window).mean() * 252 - risk_free_rate) / (downside.rolling(window=window).std() * np.sqrt(252))

rolling_sortino = rolling_sortino(daily_returns, 252)

# Calculate rolling 252-day (1 year) Calmar ratio
rolling_calmar = daily
```